

Codex + Obsidian 模块化个人知识与工作流系统设计 V2

1. 项目定位

本系统是一个以 Obsidian Vault 为持久化数据层、以 Codex 为语义整理执行器、以确定性程序为流程控制器的模块化个人知识与工作流平台。

系统本体不绑定任何具体使用场景。

澳洲硕士申请、实习经历、课程学习和日记回顾只是第一批业务模块。未来可以继续增加：

- 求职投递；
- 科研管理；
- 个人项目；
- 阅读管理；
- 会议记录；
- 旅行规划；
- 财务管理；
- 作品集管理；
- 个人健康记录。

新增业务场景时，应主要通过增加模块实现，而不是修改核心平台。

2. 总体设计目标

系统需要满足：

1. 核心平台长期稳定；
 2. 业务模块可以安装、启用、暂停和归档；
 3. 模块之间尽量不直接依赖；
 4. 每个模块可以定义自己的目录、Schema、Prompt 和工作流；
 5. 同一个模块可以创建多个实例；
 6. 用户通过文件位置提供低成本上下文；
 7. Codex 优先整理已有内容，而不是承担主要联网检索；
 8. 大部分确定性任务不调用模型；
 9. 不确定或高影响修改进入统一审核队列；
 10. 所有模块的任务统一展示在每日控制台；
 11. 历史数据在模块停用后仍可读取；
 12. 新模块接入时不需要修改核心代码。
-

3. 三层系统架构

系统分为三个层级：

核心平台



业务模块



模块实例

3.1 核心平台

提供所有模块共用的基础能力：

- 文件扫描；
- Inbox 路由；
- YAML 维护；
- 文件移动和重命名；
- 附件伴随笔记；
- 双向链接执行；
- 审核队列；
- 每日控制台；
- 周期任务；
- Git 快照；
- 操作日志；
- 模块加载；
- 权限控制；
- Codex 调用；
- 事件总线。

核心平台不应该知道：

- Monash 是什么；
- Assignment 1 应该怎样总结；
- 实习日报应该有哪些章节；
- 日记周报应该关注什么；
- 学校申请有哪些状态。

这些全部由业务模块定义。

3.2 业务模块

业务模块定义某一类场景的处理方式。

例如：

```
application-tracker
experience-log
course
journal
```

```
project-management
job-search
```

模块包含：

- 模块说明；
- 数据 Schema；
- 目录规则；
- 文件命名规则；
- Inbox 匹配规则；
- Codex Prompt；
- 工作流；
- 周期任务；
- 审核规则；
- Today 页面提供器；
- 模板；
- 可选脚本。

3.3 模块实例

模块是通用能力，实例是具体使用对象。

例如：

course 模块

```
├── COMP601-2027-S1
├── COMP602-2027-S1
└── COMP603-2027-S2
```

application-tracker 模块

```
├── australia-masters-2027
└── uk-masters-2026
```

experience-log 模块

```
├── example-company-internship-2027
└── research-assistant-2028
```

一个模块可以没有实例、拥有一个实例，或者同时拥有多个实例。

4. 系统角色分工

4.1 Obsidian

负责：

- 保存 Markdown；
- 保存 PDF、PPT、图片和其他附件；
- 浏览知识和档案；
- 手动输入内容；
- 展示 Today 页面；
- 展示审核事项；
- 提供搜索、链接和 Dataview 能力。

4.2 Codex

负责：

- 理解用户输入；
- 整理 Markdown；
- 补充语义元数据；
- 更新长期档案；
- 建议双向链接；
- 生成总结；
- 比较新旧信息；
- 生成外部检索任务；
- 解释审核原因；
- 根据用户决定执行修改。

Codex默认不是主要联网信息来源。

4.3 确定性程序

负责：

- 扫描文件；
- 检测新增和修改；
- 计算 Hash；
- 文件去重；
- 规范目录；
- 规范编号；
- 创建伴随 Markdown；
- 验证 YAML；
- 检查失效链接；
- 计算周期任务；
- 更新状态；
- 生成 Git 快照；
- 维护日志。

规则可以解决的问题，不调用 Codex。

4.4 外部联网 AI

负责：

- 查询最新招生信息；
- 查询学校官网；
- 查询学费和申请要求；
- 获取带来源的最新资料；
- 按模块要求输出结构化文档。

4.5 用户

负责：

- 记录内容；
- 将文件放入大致合适的位置；
- 审核重要或不确定修改；
- 向外部 AI 提交检索任务；
- 决定是否进行深度文档分析；
- 对最终结论负责。

5. 分层 Inbox 设计

Inbox 不再是单一的大目录，而是根据用户已经提供的上下文分层。

系统支持四种输入位置：

明确目标目录



实例 Inbox



模块 Inbox



全局 Inbox

越靠上，用户提供的上下文越明确，Token 成本越低。

5.1 明确目标目录

用户已经知道文件大致属于哪里。

例如：

```
Courses/
├── COMP601-2027-S1/
│   ├── Assignments/
│   │   ├── Homework1/
│   │   │   ├── assignment.pdf
│   │   │   ├── rubric.pdf
│   │   │   └── starter-code.zip
```

系统优先根据：

- 路径；
- 文件夹名称；
- 文件名；
- 实例配置；
- 文件扩展名；

完成：

- Homework1 规范为 A01；
- 文件重命名；
- 创建标准子目录；
- 创建伴随 Markdown；
- 自动维护 YAML；
- 更新作业索引。

默认不读取文件正文。

5.2 实例 Inbox

用户知道具体属于哪个实例，但不知道应放在哪个子目录。

例如：

```
Courses/
├── COMP601-2027-S1/
│   └── Inbox/
```

用户把课堂速记放入这里，系统已经知道：

- 模块：course；
- 实例：COMP601-2027-S1；
- 课程：COMP601；
- 学期：2027-S1。

Codex只需要进一步判断：

- 课堂笔记；
- 作业想法；

- 复习问题；
- 阅读记录；
- 课程待办。

5.3 模块 Inbox

用户知道属于哪个模块，但不知道具体实例。

例如：

```
Applications/Inbox/  
Courses/Inbox/  
Experiences/Inbox/  
Journals/Inbox/
```

以课程模块为例，系统还需要判断：

- 属于哪门课程；
- 属于哪个学期；
- 是课堂笔记还是作业资料。

以申请模块为例，系统还需要判断：

- 属于哪个申请周期；
- 属于哪所学校；
- 属于哪个专业；
- 是学费、开放状态还是材料要求。

5.4 全局 Inbox

```
00-Inbox/  
├── Quick Capture/  
├── External Imports/  
└── Needs Routing/
```

全局 Inbox 只负责：

- 快速记录；
- 无法判断模块的内容；
- 跨模块内容；
- 外部应用统一导入；
- 尚未安装对应模块的内容；
- 临时存放。

全局 Inbox 不负责系统管理。

理想状态下，全局 Inbox 应尽量保持为空。

6. Inbox 与系统管理的职责边界

必须明确区分四种系统对象。

Inbox

等待某个模块处理的输入。

Today Dashboard

等待用户执行的任务。

Review Queue

等待用户作出决定的事项。

Logs

已经发生的系统操作和事件。

对应关系：

Inbox
= 有内容等待系统处理

Today
= 有行动等待用户完成

Review Queue
= 有判断等待用户确认

Logs
= 系统已经执行了什么

系统提醒、审核任务和处理记录不应放入 Inbox。

7. Inbox 的模块声明

每个模块可以在 `module.yaml` 中声明自己的 Inbox 策略。


```
inbox:
  module_level:
    enabled: true
    path: 20-Workspace/Courses/Inbox

  instance_level:
    enabled: true
    path_pattern: "{instance_root}/Inbox"

allow_global_routing: true
```

不同模块可以使用不同策略。

课程模块

适合：

- 模块 Inbox；
- 实例 Inbox；
- 明确目标目录。

申请模块

适合：

- 模块 Inbox；
- 申请周期实例 Inbox；
- 学校或专业目标目录。

经历记录模块

适合：

- 模块 Inbox；
- 当前经历实例 Inbox。

日记模块

通常只需要：

- 日记模块 Inbox；
- Daily 直接记录入口。

不一定需要实例 Inbox。

8. 文件所有权规则

每个文件只能有一个主模块和一个主实例。

例如：

```
module: experience-log  
module_instance: company-internship-2027
```

一个文件可以关联其他模块，但不能同时由多个模块拥有和修改。

例如一篇实习日志提到课程作业：

```
主模块: experience-log  
关联模块: course
```

课程模块可以通过事件或链接接收引用，但不能直接接管该文件。

原则：

单一所有者，多模块关联。

9. 模块间通信

模块之间不直接修改彼此内部数据。

模块通过标准事件通信。

例如实习模块生成技能证据：

```
event: evidence.created  
source_module: experience-log  
entity_type: skill-evidence  
  
payload:  
  skill: API Debugging  
  source: "[[2027-03-08 实习记录]]"
```

简历模块可以订阅：

```
subscribe:  
  - evidence.created
```

课程模块也可以发布：

```
event: evidence.created

payload:
  skill: Machine Learning
  source: "[[COMP601 Assignment 1]]"
```

第一版支持以下通用事件：

```
capture.created
file.added
file.moved
note.processed
metadata.updated
review.created
review.resolved
summary.generated
research.required
deadline.approaching
evidence.created
instance.archived
```

10. Vault 目录结构

```
Personal-Knowledge-Base/
├── Home.md
├── Today.md
├── AGENTS.md
├──
├── 00-Inbox/
│   ├── Quick Capture/
│   ├── External Imports/
│   └── Needs Routing/
├──
├── 10-Sources/
│   ├── AI-Research/
│   ├── Official/
│   ├── Web-Clips/
│   ├── Emails/
│   └── Attachments/
├──
├── 20-Workspace/
│   ├── Applications/
│   │   ├── Inbox/
│   │   └── australia-masters-2027/
```

- |— Inbox/
 - |— Universities/
 - |— Documents/
 - |— Research/
 - |— Dashboard.md
 - |— Experiences/
 - |— Inbox/
 - |— company-role-2027/
 - |— Inbox/
 - |— Daily/
 - |— Weekly/
 - |— Monthly/
 - |— Projects/
 - |— Dashboard.md
 - |— Courses/
 - |— Inbox/
 - |— COMP601-2027-S1/
 - |— Inbox/
 - |— Lectures/
 - |— Assignments/
 - |— Labs/
 - |— Readings/
 - |— Notes/
 - |— Concepts/
 - |— Reviews/
 - |— Journals/
 - |— Inbox/
 - |— Daily/
 - |— Weekly/
 - |— Monthly/
 - |— Yearly/
 - |— Projects/
- |— 30-Knowledge/
- |— 80-Archive/
- |— 90-System/
 - |— Core/
 - |— Modules/
 - |— Components/
 - |— Instances/
 - |— Templates/
 - |— State/
 - |— Logs/
 - |— Review Queue/
 - |— Pending/
 - |— Approved/

```
| |—— Rejected/
| |—— Deferred/
|—— Research Requests/
|—— Processed Research/
|—— Needs Review/
|—— Scripts/
```

其中 `20-Workspace` 下的业务目录由模块创建，不由核心平台写死。

11. 模块目录标准

每个模块使用统一结构。

```
90-System/Modules/course/
|—— module.yaml
|—— README.md
|
|—— schemas/
| |—— lecture-note.schema.yaml
| |—— assignment.schema.yaml
| |—— course-instance.schema.yaml
|
|—— prompts/
| |—— classify.md
| |—— organize-lecture.md
| |—— analyze-assignment.md
| |—— weekly-review.md
|
|—— workflows/
| |—— process-capture.yaml
| |—— organize-attachment.yaml
| |—— weekly-rollup.yaml
| |—— archive-instance.yaml
|
|—— rules/
| |—— paths.yaml
| |—— naming.yaml
| |—— linking.yaml
| |—— reading-policy.yaml
| |—— review-policy.yaml
|
|—— templates/
| |—— quick-note.md
| |—— lecture-note.md
| |—— assignment.md
|
|—— dashboard/
| |—— today-provider.yaml
```

```
|  
├── migrations/  
└── tests/
```

12. 模块入口文件

`module.yaml` 是核心平台加载模块的标准入口。

```
id: course  
name: 课程学习管理  
version: 1.0.0  
status: enabled  
  
description: >  
    管理课程实例、课堂笔记、课件、作业、阅读材料和周期复习。  
  
capabilities:  
    - capture-processing  
    - metadata-enrichment  
    - attachment-management  
    - link-suggestion  
    - periodic-summary  
    - dashboard-items  
    - review-items  
  
inbox:  
    module_level:  
        enabled: true  
        path: 20-Workspace/Courses/Inbox  
  
    instance_level:  
        enabled: true  
        path_pattern: "{instance_root}/Inbox"  
  
allow_global_routing: true  
  
accepted_inputs:  
    - markdown  
    - text  
    - pdf  
    - pptx  
    - docx  
    - image  
    - archive  
  
entry_workflows:  
    capture: workflows/process-capture.yaml
```

```
attachment: workflows/organize-attachment.yaml
weekly: workflows/weekly-rollup.yaml
archive: workflows/archive-instance.yaml

review_policy: rules/review-policy.yaml
dashboard_provider: dashboard/today-provider.yaml
```

13. 模块实例结构

模块实例配置放在：

```
90-System/Instances/
```

例如：

```
90-System/Instances/COMP601-2027-S1/
├── instance.yaml
└── context.md
```

`instance.yaml`：

```
instance_id: COMP601-2027-S1
module: course
status: active

display_name: COMP601 Machine Learning
content_root: 20-Workspace/Courses/COMP601-2027-S1
inbox_path: 20-Workspace/Courses/COMP601-2027-S1/Inbox

course_code: COMP601
course_name: Machine Learning
semester: 2027-S1

created: 2027-02-20
updated: 2027-02-20
```

实例中的数据是配置，不是重新复制模块 Prompt。

14. 模块生命周期

模块生命周期：

```
installed
↓
enabled
↓
disabled
```

实例生命周期：

```
planned
↓
active
↓
paused
↓
completed
↓
archived
```

Paused

暂停：

- 自动 Inbox 处理；
- 周期任务；
- Today 提醒。

仍可以手动调用。

Completed

业务已经完成，准备生成最终总结。

Archived

停止所有自动任务，但保留：

- 数据；
- 索引；
- 搜索；
- 历史审核；
- 最终总结。

例如澳洲申请结束后：

```
status: archived
monitoring: disabled
```


无需删除申请模块或历史资料。

15. 元数据设计

15.1 核心公共字段

由核心平台维护：

```
---
id: NOTE-2027-0001

module: course
module_instance: COMP601-2027-S1
type: lecture-note

created: 2027-03-08
updated: 2027-03-08

processing_status: processed
review_status: not-required

authorship:
  user_written: true
  ai_enriched: true
  system_metadata: true
---
```

15.2 模块业务字段

由模块 Schema 定义：

```
course_code: COMP601
semester: 2027-S1
week: 3
lecture_date: 2027-03-08
topics:
  - Gradient Descent
  - Adam
```

用户不需要手动维护这些 YAML。

16. 附件元数据

PDF、PPT、ZIP 等文件通过同名伴随 Markdown 管理元数据。

```
COMP601-A01-Specification.pdf
COMP601-A01-Specification.md
```

伴随文件：

```
---
id: FILE-2027-0017

module: course
module_instance: COMP601-2027-S1
type: assignment-specification

source_file: "[[COMP601-A01-Specification.pdf]]"
original_filename: assignment.pdf
file_format: pdf
file_hash: "... "

content_read_level: 0
content_indexed: false
summary_generated: false

created: 2027-03-08
updated: 2027-03-08
---
```

YAML 由脚本和 Codex 共同维护。

17. 文件读取等级

Level 0：不读取正文

读取：

- 路径；
- 文件名；
- 扩展名；
- 大小；
- 修改时间。

默认用于明确目录中的附件。

Level 1：读取文件属性

读取：

- 标题；

- 作者；
- 页数；
- 文件内嵌属性。

Level 2：读取首页或目录

用于提取：

- 文档标题；
- 作业截止日期；
- 成绩占比；
- 课程代码；
- 文档类型。

Level 3：完整读取

仅在用户明确要求时使用：

- 整理整份课件；
- 分析完整作业要求；
- 生成复习资料；
- 比较多个文档版本。

系统不得无理由自动升级到 Level 3。

18. 输入路由流程

发现新文件



判断所在位置



明确目录	
→ 直接交给对应实例的文件整理器	
实例 Inbox	
→ 已知模块与实例，只判断类型	
模块 Inbox	
→ 已知模块，判断实例和类型	
全局 Inbox	
→ 判断模块、实例和类型	



生成修改计划



自动执行低风险操作

↓
不确定事项进入 Review Queue

19. 全局 Inbox 路由规则

全局 Inbox 中的内容先进行轻量判断。

优先使用：

1. 已有 YAML；
2. 文件名；
3. 路径；
4. 当前活跃上下文；
5. 文本前几段；
6. 模块匹配规则。

模块返回：

```
matched: true
confidence: 0.91
suggested_instance: COMP601-2027-S1
```

处理规则：

高置信度

自动移动到对应模块或实例 Inbox。

中置信度

可以暂时路由，但创建审核项。

低置信度

移动到：

```
00-Inbox/Needs Routing/
```

并在 Today 页面提示用户确认。

20. 模块接口契约

每个模块至少实现以下接口。

match

判断输入是否属于模块。

输入：

```
path:
filename:
frontmatter:
text_preview:
active_context:
```

输出：

```
matched: true
confidence: 0.91
suggested_instance: COMP601-2027-S1
```

process

生成修改计划。

```
operations:
- enrich-metadata
- normalize-markdown
- move-file
- add-link

review_items:
- ...
```

summarize

生成指定周期或阶段的总结。

```
instance: COMP601-2027-S1
period: weekly
```

dashboard

返回本模块应显示在 Today 页面中的事项。

archive

结束实例：

- 停止周期任务；
 - 生成最终总结；
 - 归档未处理内容；
 - 更新实例状态。
-

21. 双向链接机制

链接建议由模块生成，实际修改由核心 Link Engine 执行。

核心 Link Engine 负责

- 检查目标文件；
- 避免重复链接；
- 更新重命名后的链接；
- 记录链接来源；
- 控制单次链接数量；
- 将不确定链接送入审核。

模块负责

- 判断哪些对象值得关联；
- 计算业务语义置信度；
- 决定链接类型；
- 决定是否需要反向索引。

自动链接

适用于：

- 实例笔记到实例主页；
- 作业到课程；
- 研究报告到申请项目；
- 日报到经历实例；
- 周报到底层记录。

审核链接

适用于：

- 推测课堂内容与作业相关；
- 推测两篇概念笔记重复；
- 跨模块关联；
- 新建重要知识实体；
- 大规模链接修改。

22. 审核系统

Review Queue 是核心平台公共组件。

审核项格式：

```
---
review_id: REV-2027-0042
source_module: course
module_instance: COMP601-2027-S1

target: "[[COMP601 Week 03]]"
action: add-link
proposed_value: "[[COMP601 Assignment 1]]"

confidence: medium
priority: medium
status: pending
---
```

审核项记录：

- Codex 建议；
- 判断依据；
- 不确定原因；
- 用户决定；
- 执行结果。

用户可以：

- 接受；
- 修改后接受；
- 拒绝；
- 延后；
- 与 Codex 对话。

23. 审核交互方式

简单事项

直接选择：

- 正确课程；
- 正确模块；
- 是否接受链接；
- 是否合并；
- 是否延后。

简短纠正

在审核项中填写：

这是 Week 4，不是 Week 3。

Codex读取后执行。

复杂事项

点击“与 Codex 讨论”。

对话上下文只包含：

- 审核项；
- 目标文件相关片段；
- 修改建议；
- 判断依据；
- 涉及文件。

对话结束后，结论必须写回：

- 审核项；
- 目标文件；
- 操作日志。

直接修改目标文件

允许。

系统检测到文件变化后，判断是否解决已有审核项，并提示用户关闭或重新确认。

24. Today Dashboard

`Today.md` 是用户与系统最主要的交互界面。

系统级信息

- 全局 Inbox 数量；
- 无法路由内容；
- 失效链接；
- 重复文件；
- Git 快照状态；
- 系统异常。

模块级信息

每个启用模块提供自己的 Today 卡片。

例如：

Applications

- Monash AI 申请状态需要重新核验

Experiences

- 本周实习总结尚未生成

Courses

- COMP601 有一项作业关联待确认

Journal

- 本月回顾到期

待审核

按优先级统一展示：

- 高优先级；
- 中优先级；
- 低优先级。

最近完成

展示：

- 处理了哪些 Inbox；
- 更新了哪些档案；
- 生成了哪些总结；
- 移动了哪些文件；

- 建立了多少链接。

25. 外部信息检索

Codex不承担主要网页检索。

模块可以创建标准事件：

```
event: research.required
source_module: application-tracker
```

核心平台生成 Research Request：

```
90-System/Research Requests/
```

Research Request 包含：

- 当前已知信息；
- 上次核验日期；
- 需要核验的字段；
- 优先官方来源；
- 结构化输出格式；
- 发生变化时如何标记。

用户将 Prompt 交给 ChatGPT 等联网 AI。

结果放入：

```
Applications/Inbox/
```

或对应实例：

```
Applications/australia-masters-2027/Inbox/
```

申请模块负责比较和更新。

26. Prompt 分层

全局 AGENTS.md

只包含公共规则：

- 不编造事实；
- 优先路径和元数据；
- 不随意读取完整附件；
- 不随意跨模块修改；
- 高风险操作进入审核；
- 所有操作写日志；
- 用户原意不得擅自改变。

模块 Prompt

例如：

```
Modules/course/prompts/organize-lecture.md
Modules/experience-log/prompts/normalize-daily-log.md
```

实例上下文

例如：

```
Instances/COMP601-2027-S1/context.md
```

Codex调用时组合：

```
全局规则
+ 模块 Prompt
+ 实例上下文
+ 当前任务文件
```

不加载无关模块。

27. 共享组件

多个模块共用的能力放在：

```
90-System/Components/
```

建议第一批组件：

```
periodic-rollup
attachment-sidecar
status-machine
research-request
evidence-extractor
timeline-builder
comparison-table
link-suggester
```

模块声明依赖：

```
dependencies:
components:
  - periodic-rollup
  - attachment-sidecar
```

避免每个模块复制同样的 Prompt 和脚本。

28. 第一批业务模块

28.1 application-tracker

通用管理：

- 硕士申请；
- 求职申请；
- 奖学金申请；
- 博士申请；
- 实习投递。

澳洲硕士申请是一个实例或配置包：

```
application-tracker
└── australia-masters-2027
```

支持：

- 状态机；
- 截止日期；
- 材料清单；
- 外部核验；
- 对比表；
- 申请进度；

- 最终归档。
-

28.2 experience-log

通用管理长期经历：

- 实习；
- 全职工作；
- 科研助理；
- 社团经历；
- 志愿经历。

支持：

- 随手记录；
 - 日报整理；
 - 周报；
 - 月报；
 - 技能证据；
 - STAR 素材；
 - 简历素材；
 - 最终经历总结。
-

28.3 course

通用管理课程：

- 课堂笔记；
- PPT；
- PDF；
- 作业；
- Lab；
- 阅读材料；
- 周度复习；
- 考试复习；
- 概念笔记。

每门课程创建独立实例。

28.4 journal

管理：

- 每日日记；
- 周度回顾；
- 月度总结；
- 年度总结；

- 生活时间线。

该模块采用更保守的正文修改和链接策略。

29. 新模块接入流程

以后新增“求职投递”模块时：

1. 创建模块骨架

```
create-module job-search
```

2. 定义 Schema

例如：

- company;
- job-posting;
- application;
- interview;
- offer。

3. 定义 Inbox 策略

```
inbox:
  module_level:
    enabled: true
  instance_level:
    enabled: true
```

4. 编写 Prompt

例如：

- 提取岗位要求；
- 记录投递进度；
- 整理面试反馈；
- 生成求职周报。

5. 配置状态机

```
discovered
→ preparing
→ applied
```

→ interview
→ offer
→ rejected
→ withdrawn

6. 配置审核规则

例如：

- 自动识别公司名称；
- 岗位匹配度需要审核；
- 简历修改不得自动覆盖；
- 不允许自动发送申请。

7. 注册模块

核心平台扫描 `module.yaml` 后自动加载。

30. Token 控制策略

路径优先

明确目录中的内容不做全局语义分类。

模块隔离

调用一个模块时不加载其他模块 Prompt。

实例隔离

处理一门课程时不加载其他课程。

增量处理

只处理新增、修改和到期内容。

读取分级

默认从 Level 0 开始。

分层总结

原始记录
→ 周总结

- 月总结
- 年度或最终总结

上层优先读取下层总结，而不是重复读取全部原始文件。

31. 权限模型

绿色：自动执行

- 补充公共 YAML；
- 创建伴随笔记；
- 根据明确路径添加实例信息；
- 格式化 Markdown；
- 更新索引；
- 创建非破坏性总结；
- 建立明确父级链接。

黄色：执行后审核或等待审核

- 推断文件归属；
- 推断双向链接；
- 更新长期档案；
- 生成简历素材；
- 合并外部研究信息；
- 重构较大的笔记结构。

红色：事前确认

- 删除文件；
- 覆盖官方资料；
- 修改日记原文；
- 合并重要档案；
- 批量迁移；
- 修改已提交申请记录；
- Git push；
- 跨模块大规模修改。

32. 核心脚本

核心平台建议提供：

```
scan-vault
discover-modules
discover-instances
route-global-inbox
```



```
process-module-inbox
detect-new-files
generate-file-manifest
normalize-paths
normalize-numbering
create-sidecar-note
validate-frontmatter
check-broken-links
detect-duplicates
build-daily-dashboard
schedule-module-jobs
create-git-snapshot
resolve-review-item
archive-instance
```

33. Obsidian 插件界面

第一版插件只需要提供：

- 快速记录；
- 选择模块记录；
- 选择实例记录；
- 处理当前 Inbox；
- 处理全部到期 Inbox；
- 整理当前目录；
- 查看 Today；
- 查看 Review Queue；
- 生成外部检索任务；
- 生成周期总结；
- 与 Codex 讨论审核项；
- 查看最近修改；
- 撤销上一次操作。

插件不需要承担核心业务逻辑，只负责调用核心脚本和 Codex。

34. 实施顺序

阶段一：核心平台

实现：

- 模块发现；
- 实例发现；
- 分层 Inbox；
- 文件扫描；
- YAML 维护；

- Review Queue;
- Today Dashboard;
- 操作日志;
- Git 快照。

阶段二：最小模块框架

实现：

- 标准 `module.yaml` ;
- 模块接口;
- Prompt 加载;
- Schema 加载;
- Dashboard Provider;
- 生命周期。

阶段三：第一批模块

优先：

1. `application-tracker` ;
2. `experience-log` ;
3. `course` ;
4. `journal` 。

阶段四：共享组件

实现：

- 周期总结;
- 附件伴随笔记;
- 状态机;
- Research Request;
- 技能证据;
- 时间线。

阶段五：Obsidian 插件

将常用操作封装成 UI。

35. 最终系统定位

最终系统不是：

- 固定的澳洲申请知识库;
- 单一的大 Inbox 自动分类器;
- 一个 Obsidian 聊天机器人;
- 一个自动爬虫。

它是：

一个以 Obsidian 为数据层、以 Codex 为语义执行器、以确定性程序为核心基础设施，并通过可插拔模块管理不同个人场景的知识与 workflows 平台。

用户日常使用优先级为：

明确目标目录
> 实例 Inbox
> 模块 Inbox
> 全局 Inbox

用户与系统的主要交互入口为：

输入内容
+ Today Dashboard
+ Review Queue

核心平台长期保持稳定，业务需求通过模块接入，具体阶段通过实例管理。模块可以被启用、暂停和归档，而不会影响其他模块或历史数据。